



Walkthrough

Run it yourself, and know what you are looking at.

A hands-on guide. Every command below is real, every block of output was captured from an actual run, and each step says what it proves. Start to finish takes about fifteen minutes; the five-minute path is on the next page if that is all you have.

The problem, as it actually happens

It is 20:05 on a Tuesday. Payments are flowing normally — about 1,300 a minute across three gateways, three methods and eight issuing banks.

Then one gateway starts failing. Not all of it: UPI and card traffic through **gw_gamma** drops from roughly 90% success to 63%. That gateway carries 18% of volume, so on the dashboard everyone watches, the overall success rate moves about four points. Well inside a normal evening's variation.

Nobody is paged. Customers see failed payments, some retry, most leave. The money goes quietly for thirty-five minutes until somebody notices a pattern in the support queue — or until the next morning's settlement looks wrong.

That is the thirty-five minutes this project is about. What follows shows the system finding it, deciding what to do, doing it, checking whether it helped, and undoing it when it did not.

What you will be looking at

Interface	What it is	When you use it
Command line python -m razorguard.*	Nine commands. Each one runs a simulation and prints what happened. This is where the evidence lives.	Verifying the system. Recording the demo.
HTTP service :8000	The same control loop on a wall-clock timer, taking real payment outcomes over HTTP and emitting routing recommendations.	Deployment. Feeding it live data.
Console :8501	A browser view of a completed run: headline numbers, a success-rate chart, and an incident replay showing every decision as a card.	Showing somebody, rather than telling them.

If you only have five minutes

```
pip install -r requirements-dev.txt

python -m razorguard.demo           # 20s - watch one incident
python -m razorguard.experiment     # 60s - what it recovered
streamlit run app.py                #    - the same, in a browser
```

Those three answer the three questions that matter: does it detect, does the recovery figure mean anything, and does it look like a real product. The rest of this document is the long version.

Setup · once

```
git clone <your-repo-url>
cd razorguard
pip install -r requirements-dev.txt
```

That is everything for the walkthrough. Two optional extras, neither needed to follow this document:

Optional	Gives you	Without it
<code>pip install razorpay</code> + test keys in <code>.env</code>	Creating real orders in a Razorpay test account (Step 7)	Dry run prints the exact requests it would send
<code>pip install anthropic</code> + <code>ANTHROPIC_API_KEY</code>	The investigation agent answers questions in English (Step 8)	Falls back to printing the relevant ledger window

Nothing in the walkthrough fails without them. Both degrade to something useful rather than to an error.

Step 1 · Watch one incident, decision by decision

```
python -m razorguard.demo --incident INC-0-02
```

Takes about 20 seconds. You will see a timeline like this:

```
|d1 20:06 33.3% 276 #..... ! detection Success rate on gateway
                                'gw_gamma' fell 27.1 points (90.5%
                                to 63.4%). It spans 7 of 7 alarming
                                slices, 3.0x its share of the fleet,
                                so the gateway is the common factor.
> proposal shift upi/hdfc off gw_gamma
  onto gw_beta, health read at slice
  level (356 attempts over 5min)
? decision move up to 80% of the share
  gw_gamma holds onto gw_beta, which is
  running 26.7pp better
* action moved 14.4% of upi/hdfc
  from gw_gamma to gw_beta
```

Read it left to right. The bar is the health of the affected slices. The symbols are what the system did:

Sy mb ol	Stage	Meaning
!	detection	It noticed, and worked out what the failures share
>	proposal	It picked a destination and said why
?	decision	The policy engine allowed or refused it
*	action	Traffic actually moved
<	rollback	The destination turned out worse; undone
+	restore	The original healed; traffic easing back

What this proves: The system is not a threshold alarm. It found that seven alarming slices shared one gateway, at three times that gateway's share of the fleet, and said so in a sentence a human can check.

Now find the interesting lines

Scroll through the same output for these two. They matter more than the successes.

```
|d1 20:12 25.0% 52 ..... < rollback destination gw_beta fell to
```

84.1% against a 90.5% baseline on 82 attempts; reverted to baseline weights and escalated

It undid its own action. It moved traffic to gw_beta, watched gw_beta get worse, and put the weights back.

```
|d1 20:27 36.7% 79 ##..... ? decision our last 20 shifts changed
the affected keys by -1.01pp on
average, so rerouting is making things
worse rather than better. Halting all
shifts for 120 min and escalating.
rule: efficacy_breaker
```

It stopped itself entirely. Not one bad action — it measured that its whole strategy had stopped paying, and halted.

What this proves: Most systems log what they did. This one logs what it refused to do, and can conclude that it should stop. That is the part almost nothing else has.

Try other incidents

```
python -m razorguard.demo --list           # all 18 incidents
python -m razorguard.demo --incident INC-0-01 # a 3am outage
python -m razorguard.demo --incident INC-0-05 # an issuer-side fault
```

INC-0-05 is worth a look. It gets zero routing actions, and that is correct: an issuer-side fault degrades every gateway serving that bank at once, so there is no healthy destination to move to. The system recognises the signature and escalates instead of shuffling traffic between equally broken routes to look busy.

Step 2 · What did it actually recover?

```
python -m razorguard.experiment --days 2
```

About a minute. It runs the same two days twice — once with routing switched off, once on.

	router off	router on	delta

attempts	1,966,458	1,966,458	0
successful payments	1,829,522	1,840,974	+11,452
overall success rate	93.04%	93.62%	+0.58pp
revenue (Rs)	2,404,325,620	2,416,366,240	+12,040,620
Net of what the recovery cost			

gross recovered	Rs	12,040,620	
incremental processing fees	Rs	138,653	
NET recovered	Rs	11,901,967	

Look at the attempts row first. Both arms saw exactly 1,966,458 payment attempts — not approximately, exactly. That is the guard that makes the rest meaningful. If the two runs ever differ, the command refuses to print a recovery figure at all, because the comparison would be worthless.

The control arm is not crippled. It runs the identical detector and raises the identical alarms. It simply is not permitted to act.

What this proves: The number is a *difference between two measured runs*, not recovered-transactions multiplied by an average ticket. That is the difference between a measurement and a projection, and it is the thing most submissions get wrong.

The net line matters too

Rerouting is not free — acquirers price differently, so moving volume changes what the merchant pays. Gross minus fees gives net. One detail falls out of the arithmetic rather than being special-cased: **UPI carries zero MDR by regulation in India**, so a UPI recovery is free and a card recovery is not.

Step 3 · Is that one lucky run?

```
python -m razorguard.validate --seeds 8 --days 2
```

Three to four minutes; it runs eight paired trials in parallel.

	mean	sd	min	max

revenue recovered	12,291,379	593,968	11,278,430	13,117,900
share of exposure	38.2%	1.8%	35.6%	41.0%
95% CI on mean recovery: Rs 11,794,730 to Rs 12,788,027				
Seeds where the router lost money: 0/8				

What this proves: One seed cannot tell a real effect from a favourable roll of the dice. The interval excludes zero across eight independent trials, and the comparison is paired — both arms in each seed face bit-identical demand, so what is left is the policy rather than the weather.

Step 4 · Where the author was wrong

These two commands are the reason to take the rest seriously, and they are the part to show a reviewer.

```
python -m razorguard.stress --days 2
```

shift cap	recovered	of exposure	actions	rollback	congested
20%	2,131,970	6.6%	71	13	0.1%
40%	7,096,530	22.0%	91	26	0.6%
60%	9,882,790	30.6%	90	25	1.6%
80%	12,040,620	37.3%	100	32	2.6% <- shipped
100%	12,442,360	38.5%	79	33	4.3%

The traffic-shift cap was originally 40%, chosen as the cautious setting and defended in writing. This measured it: that cap was costing about **61% of the available recovery**, and the congestion it was implicitly guarding against never arrived. The default moved to 80%, the knee of the curve.

```
python -m razorguard.sensitivity --days 2
```

Around four minutes. It sweeps the cap against four different congestion curves, because the shipped one is a plausible shape rather than a measured one.

curve	20%	40%	60%	80%	100%
forgiving	6.6%	22.1%	32.4%	41.9%	43.6%
shipped	6.6%	22.0%	30.6%	37.3%	38.5%
brittle	0.2%	-2.8%	-3.0%	-4.0%	-5.1%
severe	-0.4%	-0.8%	-0.1%	-0.9%	-0.1%

Read the bottom two rows. On gateway fleets already running at capacity, this system loses money at every setting. Those fleets sit past their capacity knee before anything goes wrong, so there is no spare headroom to route into and shifting traffic only concentrates load.

What this proves: The project measures its own failure mode and publishes it. That is also why the efficacy breaker you saw in Step 1 exists — no per-action guardrail can see that the whole strategy has stopped working, so the system watches the realised effect of its own shifts and halts when they stop paying.

Step 5 · See it in a browser

```
streamlit run app.py
```

Opens at `localhost:8501`. The first run of a given set of settings takes about forty seconds while both arms execute; the result is written to `.cache/` and every load after that is about a second. The Docker image ships with the defaults already warmed.

The header is modelled on an exchange front page: a status strip with the size of the run, the board with the paired-arm quotes, a wide primary navigation, and a tape running the eighteen incidents with what each put at risk and what came back. Hover the tape to stop it. The navigation and the tiles are buttons rather than links, so clicking one reruns the page in place instead of reloading it.

Ten pages, grouped by what you are doing. Every route lives in the URL — including the simulation settings — so the tiles along the top are real links, a view can be sent to somebody as a URL, and the address bar describes the run that produced what is on screen.

- **Top row** — recovered net of fees, payments saved, actions taken, refusals, rollbacks. Click any tile to land on the page that explains it.
- **Overview** — cumulative recovery, then success rate with the router on and off. Red bands are injected incidents; triangles are actions and rollbacks. Scroll to zoom into a single incident, drag to pan, hover for the values under the pointer, and **click any point** to open that minute: the figures either side, which incident was live, and the ledger entries within three minutes of it.
- **Decision record** — a board of all eighteen incidents worst first, then the same eighteen as ranked cards, heaviest first, each carrying the decision path that produced it: what the drop was, how fast it was caught, how much money was at risk, which bounds were checked, what was done, and what it was worth. Cells tinted blue were measured against the control arm; the rest were declared before the run started.
- **Incident replay** — one incident's ledger, line by line, matched by subject as well as by time so an overlapping incident cannot lend this one its actions.
- **Recovery** — gross, the fees on it, net, and which incident each rupee came from. Negative bars are shown, not dropped.
- **Actions and Rollbacks** — every money movement, and every one the system undid after measuring the destination.
- **Refused** — every rule that stopped a money movement, how often, and what it protects against. Worth more than the successes. The engine's 93 refusals are reconciled on the page against the 57 that named a rule, so the two counters cannot be mistaken for a contradiction.
- **Audit ledger** — the full append-only record as a paged table, filterable by kind and searchable by subject, rule or text, with a CSV download.
- **How it was measured** — the paired design, the guard that refuses to report when the arms diverge, and where the model is and is not.
- **Settings** — change the seed, the simulated days, fleet traffic, and all seven policy bounds, then re-run. These are not cosmetic: they feed `WorldConfig` and `PolicyConfig` directly. Tighten the shift cap to the original 40% and gross recovery falls from Rs 1.20 cr to Rs 0.71 cr in front of you.

What this proves: The same evidence a terminal shows, in a form you can put in front of somebody who will not read a log.

Step 6 · Run it as a service

This is the deployment path — the control loop on a wall-clock timer, taking real payment outcomes over HTTP.

```
# without Docker (this path is what the tests cover)
pip install -r requirements-service.txt
export RAZORGUARD_HMAC_KEY=$(python -c "import secrets;print(secrets.token_hex(32))")
uvicorn razorguard.service:app --host 0.0.0.0 --port 8000

# or with Docker
cp -n .env.example .env      # set RAZORGUARD_HMAC_KEY
docker compose up --build
```

Then, in another terminal:

```
curl localhost:8000/health
{"ok":true,"role":"active","minute":0,"auth":"hmac",...}

# unauthenticated ingest is refused
curl -X POST localhost:8000/ingest -d '{"outcomes":[]}'
-> HTTP 401
```

Feed it aggregated counts — never individual payment records, because the detector does not need one:

```
{"outcomes":[
  {"gateway":"gw_beta", "method":"upi", "issuer":"hdfc",
   "attempts":400, "successes":150},
  {"gateway":"gw_alpha","method":"upi", "issuer":"hdfc",
   "attempts":600, "successes":580}]}
```

Endpoint	What it gives you
GET /health	Liveness, and whether this instance holds the lease
GET /routing	The output. Recommended weights per diverted key
GET /state	Minute, alarms, actions, rollbacks, ingest counters
GET /audit	The ledger from disk, refusals included
GET /metrics	Prometheus

What this proves: It is a service, not a script. State survives a restart, the ingest endpoint is authenticated with request signing, and a lease elects one active instance so two replicas cannot disagree about the fleet.

Step 7 · Create real orders in Razorpay

Optional, and free. Test-mode keys need no business verification: sign up at dashboard.razorpay.com, confirm the toggle reads **Test Mode**, then Settings → API Keys → Generate Test Key.

```
# .env
RAZORPAY_KEY_ID=rzp_test_XXXXXXXXXXXXX
RAZORPAY_KEY_SECRET=XXXXXXXXXXXXXXXXXXXXX

python -m razorguard.execute --limit 6 --live

executing 6 recoveries against Razorpay TEST mode (rzp_test_TYDp...)
[ok ] test seq=4  upi|sbi  Rs 640.00  order_TYDrGJBKKSvf3x
[ok ] test seq=8  upi|hdfc  Rs 640.00  order_TYDrGTjnLgI6Dz
...
6/6 succeeded, mode=test
```

Those are real orders. Open the Razorpay dashboard and look at one:

```
id      : order_TYDrGJBKKSvf3x    status : created
amount  : 64000 (Rs 640.00)      receipt : rg-4
notes   : razorguard_audit_seq : 4
          razorguard_subject   : upi|sbi
          razorguard_from      : gw_beta
          razorguard_to        : gw_alpha
          razorguard_reason    : moved 25.6% of upi/sbi from gw_beta
                                to gw_alpha
```

What this proves: The reasoning behind a recovery is readable from the Razorpay dashboard, outside this repository entirely. Without credentials the same command dry-runs and prints the exact requests it would send, labelled `mode=dry_run` so a placeholder can never be mistaken for a real order.

A key not beginning `rzp_test_` is refused outright, so there is no way to point this at a live merchant account by accident.

Step 8 · Ask the system why it did something

```
python -m razorguard.investigate \
    "why did traffic move off gw_beta at 03:12, and did it help?"
```

An agent with four read-only tools over the run — search the audit ledger, pull per-minute traffic for a slice, measure a key's health across every gateway serving it, rank slices worst-first. It decides for itself what to query and follows up until it can answer.

```
python -m razorguard.investigate --list-questions # suggestions
python -m razorguard.investigate --show-tools ...  # see each call
```

What this proves: Every tool reads and none write. No decision-path module can import it, and it cannot see the incident plan either — so it reasons from exactly what the system saw. Without an API key it prints the relevant ledger window instead, which is less useful than an investigation and much more useful than an error.

Step 9 · Check the claims yourself

```
pytest -q          # 202 tests, about four minutes
```

Two of those files are worth knowing about:

File	What it enforces
------	------------------

tests/test_integrity.py	No detector can import the incident plan. An Observation carries exactly six fields. No decision-path module imports a language model. No result value is hardcoded anywhere in the package. And each clause of the track's stated bar, asserted against a real run.
tests/test_documented_claims.py	Every headline figure in the README and the submission notes still matches the benchmark output. Documentation that drifts is a failing test, not something a reader has to catch.

If something goes wrong

What you see	Why	Fix
docker: not recognized	Docker Desktop is not installed	Skip it. The uvicorn path in Step 6 is the one the tests cover.
docker-credential-desktop not found	Docker's helper is not on PATH after a fresh install	Open a new terminal, or prepend \$env:ProgramFiles\Docker\Docker\resources\bin
refusing to run against a key that is not test mode	You generated a live key	Switch the dashboard toggle to Test Mode and regenerate
set RAZORPAY_KEY_ID and RAZORPAY_KEY_SECRET	No .env found, or it is named wrong	The file must be exactly .env in the project root
Service will not start	No credential configured — deliberate	Set RAZORGUARD_HMAC_KEY. Open is something you opt into, not something you forget.
Narration is plain, not written by a model	No ANTHROPIC_API_KEY	Expected. It falls back to a deterministic template, and no result depends on the model having run.

What is simulated, and what is not

Worth being clear about before you draw conclusions from any number above. **The world is invented; the measurement is real.**

Invented	Measured
Gateway names, traffic mix, volumes, the daily curve, healthy success rates, average ticket sizes, the congestion curve, and the incident schedule	Detection latency, false-alarm rates, recovered payments, success-rate gain, processing cost, and every confidence interval

The rupee figures scale with invented ticket sizes. A sceptical reader should take the result as **+11,452 successful payments** and **+0.58 percentage points** of success rate, neither of which depends on a price. DATA.md lists every invented constant line by line.

The reason invented incidents make the measurement *more* trustworthy rather than less: nothing here learns a label. The outages are scheduled, so time-to-detect is latency against an event we caused — not agreement with a label we invented. The detector sees only (slice, minute, attempts, successes), and a test asserts it cannot import the incident plan.

Where to go next

Document	For
README.md	The short version, with the architecture diagram
RazorGuard-Complete-Documentation.pdf	Why each decision was made, and the ones measurement overturned
RazorGuard-Summary-and-Workflow.pdf	The five-page version
RazorGuard-Pitch-3min.pdf	What to say on camera, and what to have on screen while saying it
RazorGuard-Pitch-6min.pdf	The six-minute version, with room for a worked example
RazorGuard-Voiceover-10min.pdf	The same, at walkthrough length, segment by segment
DEPLOY.md	Running it against real traffic, and what is still open
DATA.md	Every invented constant, and which results depend on it